Press Play with Newt

Triscuitcircuit

October 21, 2026

Contents

1 One-time setup on a developer Mac **2**

2 Add the plugin to a game **2**

3 Handoff behavior **3**

4 Terminal and development app bundling **4**

5 Self-contained standalone bundles **4**

6 Validation **5**

Newt Play Bridge is a Godot 4 editor plugin for macOS. It discovers the local user's managed Newt session over a private Unix socket. Pressing Play reserves the headset, fades/stops the dashboard, configures the game's runtime environment, and lets Godot start the game normally. Newt watches the game process and restores the dashboard after Stop or a crash.

1 One-time setup on a developer Mac

See Install Newt and locate its libraries. You can now configure paths from **Project → Tools → Configure Newt...** in Godot using file pickers. Save once before pressing Play.

Build and install Newt, including `newt-demo`, and supply a compatible macOS OpenXR runtime. Configure the installed developer command once:

```
newt configure --dashboard /path/to/newt-demo --runtime /path/to/runtime.json --godot /
```

Add `--icd /path/to/MoltenVK_icd.json` if the runtime needs an external Vulkan ICD. This is machine configuration, not game code. The runtime and all of its dynamic libraries must already be usable on this Mac; this command does not build or download a runtime. Configuration is stored in `~/Library/Application Support/Newt/config.json`.

From a source checkout, use the built `build/newtd` instead of the installed `newt` command (same binary; `newt` is a symlink to `newtd` once installed).

2 Add the plugin to a game

Copy `addons/newt_bridge` into the project's `addons` directory, or run:

```
python3 tools/install-godot-plugin.py /path/to/godot-project
```

1. Enable **Newt Play Bridge** in Project Settings → Plugins.
2. Enable OpenXR and XR shaders. Choose the Mobile renderer and Vulkan for the macOS rendering device. Restart the editor after changing renderer/XR settings.
3. In Project Settings → Newt, set `executable` if `newtd` is not found via `~/.local/bin`, `/usr/local/bin`, `/opt/homebrew/bin`, or `PATH`. `auto_start` defaults to true; turn it off to require an existing session.

4. Press Play (F5/F6). No runtime manifest path or Newt API call is needed in game scripts.

The plugin talks to the installed native `newtd` daemon; no Python is involved at editor or game runtime. It uses the same user configuration and IPC endpoint as the installed Newt command. It is development tooling; it does not embed a runtime into the game or add a second XR renderer.

3 Handoff behavior

- One headset owner at a time. A running game rejects a competing launch.
- Dashboard stop gets a graceful fade window, then bounded terminate/kill fallback before the game starts.
- A one-use reservation is passed to the child through its environment. A tiny autoload registers the game PID; no modifications to game scenes are required.
- The manager restores the dashboard only after the game process exits, avoiding a race with OpenXR teardown. A crash also releases the headset.
- An unclaimed reservation expires after 60 seconds, restoring the dashboard if startup fails or another editor plugin cancels Play.
- A dashboard that exits does not enter an automatic crash/reconnect loop. `newt resume` retries it.
- The manager stays alive, but the dashboard helper restarts. Its current session-only screen layout/settings are not preserved by this handoff yet.
- Discovery is local, per user. An independently launched `newt-demo`, an unrelated OpenXR app, or another product's dashboard is not automatically adopted. Start Newt through `newt start` or let this plugin start its managed session. Close any independently running headset owner first.
- Editor-launched games remain owned by Godot. `newt stop` stops the manager and its own children; use Godot Stop to end an editor-owned game.

Useful commands: `newt start`, `newt status`, `newt resume`, `newt stop`.

4 Terminal and development app bundling

```
newt run --project /path/to/godot-project
newt run --game /path/to/ExportedGame.app
newt bundle --game /path/to/ExportedGame.app --output /path/to/Game-Newt.app
```

`run` starts or reuses Newt, hands off to the game, then restores the dashboard. `bundle` creates a new development app containing the unchanged exported game plus a small native launcher. Opening it submits the game to the same manager. It reuses the configured runtime rather than duplicating runtime libraries in every game. The source export is not modified and existing outputs are refused.

This development bundle requires the configured Newt installation on that Mac. (Creating a standalone bundle shells out to `python3` for the dylib-closure step, on the developer Mac only; the produced app needs no Python.) It is not a self-contained distributable or notarized release. A production installer/native manager with runtime version selection and a licensed runtime distribution is a separate packaging milestone. Bundling does not automatically export a Godot project; use Godot's macOS export first.

5 Self-contained standalone bundles

For distribution to Macs that have never installed Newt, `bundle` also has a standalone mode that embeds a developer-supplied OpenXR runtime (and optional Vulkan ICD, e.g. MoltenVK) with their complete dylib closures, plus the `newtd` manager itself:

```
newt bundle --game /path/to/ExportedGame.app --output /path/to/Game.app \
  --runtime /path/to/runtime.json --icd /path/to/MoltenVK_icd.json \
  --bundle-id com.example.mygame
```

Newt does not supply runtime binaries; you provide locally built ones, as above. The output app's launcher points `newtd` at a private state directory (`~/Library/Application Support/<bundle-id>/Newt`) and a matching private IPC directory, and the first launch auto-configures itself from the embedded manifests — no `newt configure`, dashboard, or Pippin needed on the player's Mac, and a separately installed Newt/Pippin setup is never

touched. Quitting the game stops the manager too; nothing lingers. Signing uses `$CODESIGN_IDENTITY` (ad-hoc by default). See Self-contained Godot export for the design.

To produce this automatically from **Project → Export**, set Project Settings → Newt → Distribution: `runtime_manifest` (required to activate), `icd_manifest` (optional), and `bundle_id` (defaults to `org.newt.<your project name>`). After each macOS `.app` export, the plugin writes `<name> (Newt-ready).app` next to it. Leaving `runtime_manifest` empty disables export-time bundling entirely.

6 Validation

Hardware-free regression tests cover launch ordering, runtime inheritance, exclusive ownership, game exit/failure, invalid reservation tokens, external-game completion, startup reservation expiry, duplicate managers, and shutdown. A headless Godot test also exercises the actual plugin build hook, inherited environment, and game autoload. The native development app bootstrap is compiled, signed, and tested against a fake exported app. Full F5-to-headset rendering and return still require an on-device test of the chosen runtime/engine combination.

References: Godot EditorPlugin build hook, Godot process environment.